

# 架构 — Agent + Skills 的逻辑

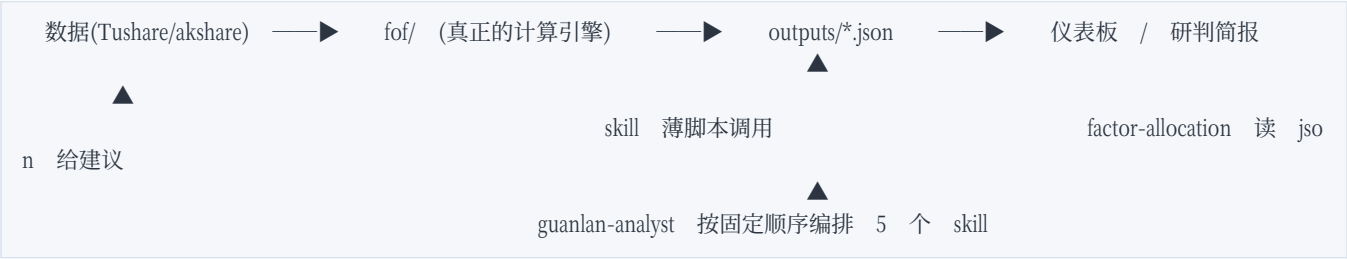
本文解释 `.claude/` 下 1 个 agent + 5 个 skill 与 `fof/` 引擎如何分工协作。配套阅读：

[README.md](#)（项目总览）、[CLAUDE.md](#)（硬约束）。

## 一句话总览

Skill = 一块能力（触发说明 + 薄脚本 + 读法），Agent = 把这些能力按固定顺序串起来的编排者。

所有计算都在 `fof/` 这个 Python 包里；skill 只是门面——薄脚本调 `fof/` 的函数、写一个 `outputs/*.json`、打印一行摘要。



## 一、分层逻辑（为什么这么分）

| 层       | 是什么                              | 放哪                                             | 改它的代价   |
|---------|----------------------------------|------------------------------------------------|---------|
| 计算层     | HMM / ER / 因子 / 指标的真实算法          | <code>fof/*.py</code>                          | 重，必须配单测 |
| Skill 层 | 触发描述 + 薄脚本(薄包装) + references(读法) | <code>.claude/skills/&lt;name&gt;/</code>      | 轻，门面    |
| Agent 层 | 固定 workflow，串 skill，产带溯源简报       | <code>.claude/agents/guanlan-analyst.md</code> | 轻，编排    |

核心设计：逻辑全在 `fof/`，skill 脚本绝不写算法。好处是——防前视纪律、JSON 安全、密钥安全只在一处保证；skill 换了/删了，算法和证据都不动（这正是 FOF 模块来回增删都没伤到引擎的原因）。

## 二、5 个 Skill 各自的「一件事」

每个 skill = 一个 durable job，边界清晰、互不重叠（`SKILL.md` 里的 NOT 段落防误触）。

| Skill           | 输入                 | 输出                                | 本质                          | 类型  |
|-----------------|--------------------|-----------------------------------|-----------------------------|-----|
| regime-radar    | 12 个真实指标           | <code>outputs/regime.json</code>  | 0-100 风险分 + 顶/底双评           | 生产者 |
| regime-verdict  | HMM 4 态 × ER × 基本面 | <code>outputs/master.json</code>  | 大势研判 走强/震荡/走弱               | 生产者 |
| factor-research | 12 风格因子            | <code>outputs/factors.json</code> | 因子看板 排行/IC · ICIR /滚动Sharpe | 生产者 |

| Skill                    | 输入                 | 输出                             | 本质              | 类型  |
|--------------------------|--------------------|--------------------------------|-----------------|-----|
| factor-allocation        | 读 master + factors | outputs/factor_allocation.json | 配置建议 姿态 + 超配/低配 | 消费者 |
| quant-research-retriever | vault 研报库          | 引用 JSON                        | 溯源 找论据          | 工具  |

生产者 vs 消费者——这是「集成」的关键：

- 生产者（前 3 个）：薄脚本调 fof/ 算法 → 自己 fetch 数据、算指标 → 写 json。
- 消费者 factor-allocation：不碰数据，只读 master.json（大势）+ factors.json（因子）→ 映射成建议。它站在另外两个 skill 的肩上 —— 这就是「两个集成 skill」里的集成。

每个 skill 目录都是同一个三件套：

```
<skill>/
├── SKILL.md                ← frontmatter.description 决定「什么时候被触发」（最关键的触发面）
├── scripts/xxx.py          ← 薄包装，真正逻辑在 fof/
└── references/xxx.md       ← 「怎么读结果」的深度说明，按需加载，不挤占 SKILL.md
```

### 三、触发逻辑（skill 怎么被调起来）

SKILL.md 的 description 是触发面——它把用户的话和 skill 匹配。三种触发路径：

- 自然语言 → Claude 匹配 description 触发语，自动加载 SKILL.md 执行
  - 「现在走强还是走弱」→ regime-verdict；「该超配哪些风格」→ factor-allocation
- Agent 自动调 → guanlan-analyst 工作流里按步骤调（用户无感）
- 命令行直接跑 → python .claude\skills\

每个 SKILL 的 NOT 段落是为了防误触：factor-allocation 明说「不建组合/不回测」，就不会与 factor-research（看因子）/ regime-verdict（判大势）抢触发。

### 四、Agent 的逻辑（编排 = 固定顺序 + 纪律）

guanlan-analyst 自己不算任何东西，它是胶水 + 纪律。固定工作流：

```
1. GROUND      → quant-research-retriever    先查研报，≥2 条 Brain 路径引用（先取证、再下结论）
2. COMPUTE     → scripts/run_pipeline.py      一次产 regime+master+factors 三 json（防前视、免重算）
3. READ        → 用 3 个 skill 的 references  读法解读三份 json
4. BRIEF       → 带溯源研判简报：大势 verdict + 风险 band + 因子读数 + 一句姿态(进攻/中性/防御)
5. ADVISE(可选) → factor-allocation           用户要「配置建议」时才调
```

Agent 的价值不在「会算」，而在强制纪律：

- never invent numbers —— 每个数字来自 json，不许编。
- 每个机制配 ≥2 条研报引用 —— 不许空口断言。
- 诚实 —— 因子动量弱(IC≈0.04)不许吹成 alpha；大势历史色带是 walk-forward 样本外；ER 在涨跌停虚高。
- no-key resilience —— 整条 pipeline + 两页仪表板无 ANTHROPIC\_API\_KEY 也能跑；仅仪表板 live chat 需要 key，agent 自身从不调 Anthropic API。

## 五、贯穿所有层的 4 条硬约束（逻辑的地基）

这 4 条只在 **fof/** 保证一次，**skill/agent** 自动继承（详见 [CLAUDE.md](#)）：

- 1. 防前视：决策只用 `series.loc[:asof]`，回测收益 `nav.pct_change().shift(-1)`（收盘决策、次日兑现）；HMM 每月 walk-forward 只用 `[:m]` 重拟合，不含未来信息。
- 2. 密钥安全：TUSHARE\_TOKEN / ANTHROPIC\_API\_KEY 只从 `gitignored .env` 读，绝不进日志/提交。
- 3. JSON 安全：所有 `outputs/*.json` 经 `report._finite` 清洗 NaN/Inf → null（Starlette JSONResponse 拒绝 NaN）。
- 4. 诚实交付：弱信号（因子动量  $IC \approx 0.04$ ）如实标注；不构建组合、不回测、不出权重——交付是研究建议与读法，非可上线策略。ETF-FOF 早期回测脚手架仅作历史代码参考留在 **fof/**，不在文档呈现。

## 六、端到端示例

用户：「现在该进攻还是防御、配什么？」

guanlan-analyst：① 查研报锚定机制（HMM regime / 因子动量， $\geq 2$  条 Brain 引用）→ ② 跑

run\_pipeline.py 出三 json → ③ regime-verdict 给「震荡 50.7」、factor-research 给因子排行+IC →

④ factor-allocation 读这两份 → 输出「震荡市 → 中性半仓，动量 tilt 超配反转/低波/红利」，

每条带 Brain 引用、带诚实 caveat（因子动量弱、这是研究建议非组合）。

一句话收尾：**fof/** 负责「算得对」（防前视 + 证据），**skill** 负责「调得动」（触发 + 读法），**agent** 负责「串得顺、说得诚实」（编排 + 纪律）。

⚠ 仅供量化研究参考，不构成投资建议。